

Trabajo Práctico Integrador Final

En el presente trabajo integrador se busca que los alumnos apliquen los conocimientos adquiridos a lo largo del curso, apoyándose en prácticos anteriores y la bibliografía provista.

Al igual que el trabajo práctico anterior, se realizará en grupos manteniendo la misma conformación.

En 2 mundos de Gazebo elegidos por el grupo, deberán mapearlos utilizando el modelo de robot del trabajo anterior y técnicas de **SLAM**.

Luego deberán aplicar **path planning** sobre este mapa, permitiendo la navegación del robot entre 2 puntos cualesquiera del mismo. La trayectoria generada será convertida en una serie de comandos de velocidad para el robot con un **motion planner**.

La configuración e interconexión de estos sistemas se hará mediante un **nodo orquestador** a diseñar por el grupo: deberá recibir la pose de meta deseada, pasársela al planner para que calcule una ruta hasta ella, luego al controller para que convierta la trayectoria en comandos de velocidad y los publique.

Finalmente, se deberá configurar un robot físico para que reciba y ejecute los comandos de velocidad generados.

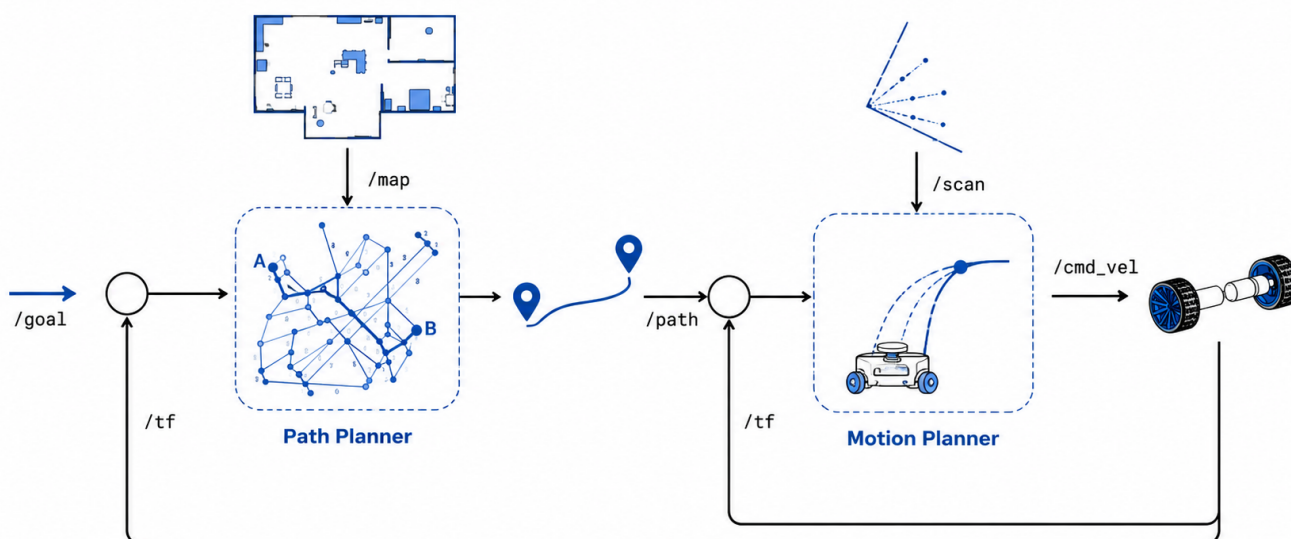


Figura 1: Interacción entre el Planner y el Controller

A continuación se presenta un desglose de las actividades a realizar junto a los requerimientos.

Parte 1: SLAM

Preparar 2 mundos distintos en Gazebo que presenten un desafío para el mapeo y navegación del robot (diversos obstáculos, ambigüedades espaciales). Pueden obtenerse de los repositorios recomendados en clase o ser creados por el mismo grupo.

Utilizando técnicas de SLAM (`slam_toolbox`), construir un mapa de los mundos elegidos y guardarlo como archivos `.pgm + .yaml` mediante `nav2_map_server`. Para poder usar `slam_toolbox` deberán configurar el nodo apropiadamente (se recomienda partir de la configuración predeterminada del `toolbox` y adaptar según sea necesario).

Para los mapas, seleccionar un tamaño de celda que presente un balance adecuado entre resolución y costo computacional.

El lanzamiento y configuración de todos los nodos debe realizarse con un único launch file, y la gestión de los nodos de ciclo de vida debe realizarse con `nav2_lifecycle_manager`.

Parte 2: Localización

Generar un launch file que lance una simulación con el robot, el mapa obtenido con SLAM (se carga como archivo con `nav2_map_server`) y **localización global** usando `nav2_amcl`. Los parámetros del nodo de localización deberán configurarse también con un archivo `yaml`.

Esta parte servirá como validación de lo realizado en la etapa anterior, donde se debe poder visualizar en RViz al robot desplazándose por el mapa obtenido.

Parte 3: Costmap, Planner y Controller

A partir de los mapas construidos en la primera parte, configurar un **costmap** con `nav2_costmap_2d` con capa estática, de inflación y de obstáculos. La capa estática usará como entrada el mapa obtenido previamente, y publicado por el `map_server`. La capa de obstáculos recibirá las lecturas del LIDAR. La capa de inflación debe configurarse según el tamaño del robot y el margen de seguridad deseado. Los parámetros para el costmap en general y para cada capa en particular estarán definidos en un archivo de configuración propio del mismo.

Configurar el path planner con su global costmap para que se pueda usar **A* o Dijkstra** (se recomienda `Navfn_planner` o `SmacPlanner2D`).

Configurar el motion planner con su global y local costmaps, para que pueda correr 2 algoritmos distintos (se recomiendan **DWB, RPP o MPPI**).

El mecanismo para la selección de algoritmos queda a criterio del grupo (por ej. se puede definir desde el launch file o cambiarlo en tiempo de ejecución mediante un servicio).

Parte 4: Orquestador

En esta etapa se unirán los nodos de path y motion planning mediante un nodo orquestador, permitiendo **indicarle metas al robot** de forma interactiva y visualizando su progreso. La pose de meta se indicará utilizando el "2D Goal Pose" de RViz, y el robot deberá ser capaz de navegar a cualquier punto físicamente posible del mapa.

El nodo debe:

- Suscribirse al tópico `/goal_pose` publicado por la herramienta '2D Goal Pose' de RViz.
- Consultar la pose actual del robot vía TF (`map → base_link`).
- Ser action client de `ComputePathToPose` (Nav2 Planner), enviar la meta, el goal checker y el algoritmo a utilizar como Goal, y recibir el path como Result.

- Ser action client de FollowPath (Nav2 Controller), enviando el path obtenido como Goal para que el controller lo ejecute.
- Replanificar periódicamente a una frecuencia configurable mientras el robot navega, actualizando el path del controller.
- Detectar la finalización: reportar por log cuando la meta fue alcanzada (/follow_path retorna éxito) o cuando la navegación falló, indicando en qué etapa.
- Manejar una meta nueva que llegue mientras hay una navegación en curso: cancelar la acción activa y comenzar la nueva.

Parte 5: Implementación en robot físico

El robot físico deberá recibir y ejecutar los comandos de velocidad generados por el motion planner, de forma que se mueva de forma muy similar a la simulación. El grupo deberá evitar que comandos de velocidad de otros robots interfieran con la operación del suyo.

No se requiere que el robot realice una evasión de obstáculos, simplemente que siga los comandos de velocidad.

Entregables

- Archivos .sdf/.world y captura de pantalla de los mundos utilizados.
- Archivos .pgm + .yaml y captura de pantalla de los mapas construidos.
- Archivos de configuración de slam_toolbox, nav2_amcl, los costmap, planner y controller.
- Launch files utilizados para el setup de la simulación.
- Capturas de RViz mostrando el robot realizando SLAM y localizándose en el mapa ya conocido (Parte 1 y 2).
- Captura del costmap generado (Parte 3).
- Captura de RViz mostrando la trayectoria calculada en base al Goal seleccionado, y el robot siguiéndola (Parte 5).

Bibliografía

- Diapositivas Clases 4-12
- [Repositorio/Documentación de slam_toolbox](#)
- [Configuración de parámetros de slam_toolbox](#)
- [Documentación Oficial Nav2](#)
- [Configuración del costmap 2D](#)
- [Parámetros Costmap 2D](#)
- [Navigation Plugins \(Planner and Controller Servers\)](#)
- [Planner Plugin Configuration](#)

- Thrun, Burgard & Fox, *Probabilistic Robotics*, caps. 7–10
- *Modern Robotics: Mechanics, Planning, and Control* (Lynch & Park)
- *A Concise Introduction to Robot Programming with ROS2* (F. M. Rico)